— *Solution notes* —

COMPUTER SCIENCE TRIPOS Part IB – 2023 – Paper 5

4 Concurrent and Distributed Systems (djg11)

Synchronous message passing using semaphores

(*a*) A guarded resource needs to be locked in three different ways by readers, moderators and writers. A thread will gain access to the resource in one of these three ways, operate, and then relinquish access. At any instant, the resource may be unlocked or held be any number of readers, or up to two moderators, or at most one writer.

(*i*) Define some number of locks, mutexes and/or shared variables to manage the system. Sketch the core of a state transition diagram. [5 marks]

Answer: [*No requirement was requested to avoid starvation, so an ultra-simple answer is possible.*] An enum STATE ranges over four states: an IDLE state and three satellite states, READERS, WRITER, MODERATORS, each with a pair of arcs to and from the IDLE state. This is cross producted with a counter that ranges between 1 and 2 when state is MODERATORS and is any natural number when state is READERS. [*A candidate could attempt to write out the full cross product but will need an ellipsis to support an unbounded reader count. One example full diagram is listed below, but is far more than needed for the 'core'.* Two MODERATORS states could be used instead of using the counter for moderators, but it makes the code more complicated.] One further binary mutex can be used to provided exclusion, but this will be intrinsic in the following monitor and need not be listed. Ditto the thread queues for the monitor mutex and condition variable.

(*ii*) Using a monitor approach, give pseudocode for these six methods:

```
start_write()   start_moderate()   start_read()
end_write()     end_moderate()     end_read()
```

[5 marks]

Answer: See last page.

deadlock

(*b*) Very briefly describe two techniques for deadlock avoidance and one technique for graceful deadlock recovery (i.e. not reboot). Describe two burdensome aspects of deadlock recovery. [6 marks]

Answer: Deadlock can be avoided by taking out locks in a system-wide order. This is a static approach. Alternatively, the Banker's Algorithm is a dynamic approach that avoids deadlock by deferring allocation of a resource if it might be needed to enable another customer to complete their business. Deadlock recovery typically involves 'killing' a customer, but to be graceful, the customer must have exception handling code that allows it to retry, perhaps after a delay. *NB*: clients of a transaction processing system are always coded robustly w.r.t. enforced aborts. Two problems are detecting deadlock (using perhaps some mixture of time-outs and dependency graphing) and undoing changes already made by an aborting transaction (rollback).

(*c*) A system has T threads. These use 3 types of resource that each have 4 instances provided. A deadlock avoidance system restricts resource acquisition. What state space does the system potentially have before and after restriction? [4 marks]

Answer: In terms of allocation state, there are 12 resources and each could be idle or in use by one of the T threads, so a 12-location array of thread identifiers (tids) simply serves, with zero being banned as a tid and instead being to used to denote that a resource is free. Deadlock avoidance does not need to know which specific resource is held by which thread, so this encoding is possibly richer than strictly needed, but the full information needs to be held somewhere for system operation.

There are numerous deadlock avoidance systems, but the majority of them, such as ordered locking or Banker's, make no restriction on which resources can be held by which threads. To observe a reduction in state space, the program counters of the participants need to be included (preferably abstracted to a level that excludes thread-local behaviour (as covered in the semantics course). For instance, the Banker's algorithm can use a model of the program flow that reflects the total numbers of further resources possibly claimed from the current execution point, or a simplification of it (*in extremis* just the total number ever needed from the program entry point!). [*Just four of the points above are needed for full credit.*]

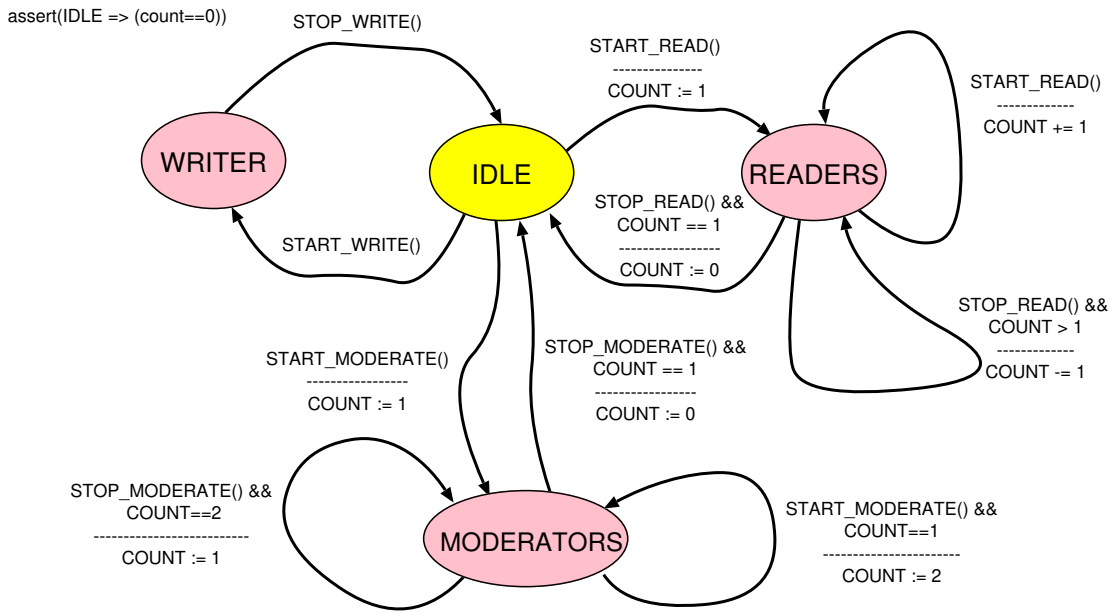
Answer:

```
monitor lock3()
{
  enum { IDLE, READERS, WRITER, MODERATORS } state = IDLE;
  condition_var waiters;   int count = 0;

  void start_write() { while (state != IDLE) wait(waiters);  state = WRITER; }
  void end_write()   { STATE = IDLE;  signal_all(waiters); }

  void start_read() {
    while (state != IDLE && state != READERS) wait(waiters);
    count += 1;
    state = READERS; }
  void end_read()   { if (--count == 0) STATE = IDLE; signal_all(waiters); }

  void start_moderate() {
    while (state != IDLE || (state == MODERATE && count==1)) wait(waiters);
    count += 1;
    state = MODERATE; }
  void end_moderate() { if (--count == 0) STATE = IDLE;  signal_all(waiters); }
}
```


assert(IDLE => (count==0))
STOP_WRITE()
START_READ()

COUNT := 1
START_READ()

COUNT += 1
WRITER
IDLE
READERS
START_WRITE()
STOP_READ() &&
COUNT == 1

COUNT := 0
STOP_READ() &&
COUNT > 1

COUNT -= 1
START_MODERATE()

COUNT := 1
STOP_MODERATE() &&
COUNT == 1

COUNT := 0
STOP_MODERATE() &&
COUNT==2

COUNT := 1
MODERATORS
START_MODERATE() &&
COUNT==1

COUNT := 2